

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **ARTIFICIAL INTELLIGENCE**

Subject Code: **CS T71**

UNIT IV

Planning and Learning: Planning with state space search-partial order planning-planning graphs-conditional planning-continuous planning-Multi-Agent planning. Forms of learning-inductive learning-learning decision trees-ensemble learning-Neural Net learning and Genetic learning

Unit IV- Two Marks

1. What is planning in AI? (Apr'15) (Nov'15)

Planning is done by a planning agent or a planner to solve the current world problem. It has certain algorithms to implement the solutions for the given problem and this is called ideal planner algorithm

(i) How does a planner implement the solution?

Unify action and goal representation to allow selection (Use logical language for both)

Divide-and-conquer by subgoaling

Relax requirement for sequential construction of solutions

(ii) What is the difference between planner and problem solving agents?

Problem solving agent: It will represent the task of the problem and solves it using search techniques and any other algorithm.

Planner: It overcomes the difficulties that arise in the problem solving agent.

4. What are the effects of non- planning? (Apr'15) (Nov'15)

infinite branching factor in case of many tasks

choosing heuristic function and works in the same sequence/order

5. What are the key ideas of planning approach?

3 key ideas to approach planning are open up the representation of states, goals ,actions using FOL the planner is free to add actions to the plan whenever and wherever they are needed rather than an incremental sequence most part of the world are going to be independent of the other parts

6. State the 3 components of operators in representation of planning?

(i) Action description

(ii) Pre-condition

(iii) Effect

7. What is STRIPS?

Stanford Research Institute Problem Solver

1. Tidily arranged actions descriptions
2. Restricted language (function-free literals)
3. Efficient algorithms

8. What is situation space planner?

The planner takes the situation and searches for it in the KB and locates it. It is reused if it is already present else a new plan is made for the situation and executed.

1. What are the drawbacks of programmer's planner?

- High branching factor during searching
- Huge search space

2. What are the Planning Algorithms for Searching a World Space? Searching a World Space:

There are two algorithms:

Progression: An algorithm that searches for the goal state by searching through the states generated by actions that can be performed in the given state, starting from the initial state.

Regression: An algorithm that searches backward from the goal state by finding actions whose effects satisfy one or more of the posted goals, and posting the chosen action's preconditions as goals (*goal regression*).

11. What is partial plan?

Partial plan is an incomplete plan which may be done during the Initial phase. There are 2 main operation allowed in planning

- Refinement operator
- Modification operator
- (Partial) Plan consists of
- Set of operator applications S_i
- Partial (temporal) order constraints $S_i \rightarrow S_j$
- Causal links $S_i \xrightarrow{c} S_j$

12. What is fully instantiated plan? (Apr'15) (Nov'15)

A fully instantiated plan is formally defined as a data structure consisting of the following 4 components

- A set of plan steps
- A set of step monitoring constraints
- A set of variable binding constraints
- A set of casual links

13. What is complete plan?

A plan is complete iff every precondition is achieved A precondition c of a step S_j is achieved (by S_i) if

$$S_i < S_j$$

$$c \in \text{effect}(S_i)$$

there is no S_k with $S_i < S_k < S_j$ and $\neg c \in \text{effect}(S_k)$ (otherwise S_k is called a clobberer or threat)

14. What are the Properties of POP?

Non-deterministic search for plan, backtracks over choice points on failure:

- . choice of *Sadd* to achieve *Sneed*
- . choice of promotion or demotion for clobberer Sound and complete

There are extensions for:

Disjunction, universal quantification, negation, conditionals, Efficient with good heuristics from problem description but: very sensitive to sub goal ordering, good for problems with loosely related sub goals

15. What is conditional planning?

Conditional planning

Plan to obtain information (observation actions) Subplan for each contingency

Example: [Check(Tire1); If(Intact(Tire1); [In_ate(Tire1)]; [CallHelp])] Disadvantage: Expensive because it plans for many unlikely cases

Similar to POP If an open condition can be established by observation action

Add the action to the plan

Complete plan for each possible observation outcome

16. What is monitoring/ replanning

Monitoring / Replanning Assume normal states / outcomes

Check progress during execution, re plan if necessary Disadvantage: Unanticipated outcomes may lead to failure.

17. Distinguish between the learning element and performance element

Learning element is responsible for making improvements.

The learning element takes some knowledge about the learning element and some feedback on how the agent is doing, and determines how the performance element should be modified to (hopefully) do better in the future.

Performance element is responsible for selecting external actions.

The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions.

18. What is the role of critic in the general model of learning?

The **critic** is designed to tell the learning element how well the agent is doing. The critic employs a fixed standard of performance. This is necessary because the percepts themselves provide no indication of the agent's success.

19. Describe the problem generator.

It is responsible for suggesting actions that will lead to new and informative experiences. The point is that if the performance element had its way, it would keep doing the actions that are best, given what it knows. But if the agent is willing to explore a little, and do some perhaps suboptimal actions in the short run, it might discover much better actions for the long run.

20. What is meant by Speedup Learning? (Apr/May'14) (Nov/Dec'14) (Apr'15) (Nov'15).

The learning element is also responsible for improving the *efficiency* of the performance element. For example, when asked to make a trip to a new destination, the taxi might take a while to consult its map and plan the best route. But the next

time a similar trip is requested, the planning process should be much faster. This is called **speedup learning**

21. What are the issues in the design of learning agents?

The design of the learning element is affected by four major issues:

1. Which *components* of the performance element are to be improved?
2. What *representation* is used for those components?
3. What *feedback* is available?
4. What *prior information* is available?

22. List the Components of the performance element

5. A direct mapping from conditions on the current state to actions.
6. A means to infer relevant properties of the world from the percept sequence.
7. Information about the way the world evolves.
8. Information about the results of possible actions the agent can take.
9. Utility information indicating the desirability of world states.
10. Action-value information indicating the desirability of particular actions in particular states.
11. Goals that describe classes of states whose achievement maximizes the agent's utility.

23. What are the types of learning?

- i. Supervised learning

ii. Unsupervised learning

iii. Reinforcement learning

24. What are the approaches for learning logical sentences?

The two approaches to learning logical sentences:

Decision tree methods, which use a restricted representation of logical sentences specifically designed for learning,

Version-space approach, which is more general but often rather inefficient.

25. What are decision trees?

A **decision tree** takes as input an object or situation described by a set of properties, and outputs a yes/no "decision." Decision trees therefore represent Boolean functions. Each internal node in the tree corresponds to a test of the value of one of the properties, and the branches from the node are labelled with the possible values of the test. Each leaf node in the tree specifies the Boolean value to be returned if that leaf is reached.

26. Give an example for decision tree logical sentence.

The path for a restaurant full of patrons, with an estimated wait of 10-30 minutes when the agent is not hungry is expressed by the logical sentence

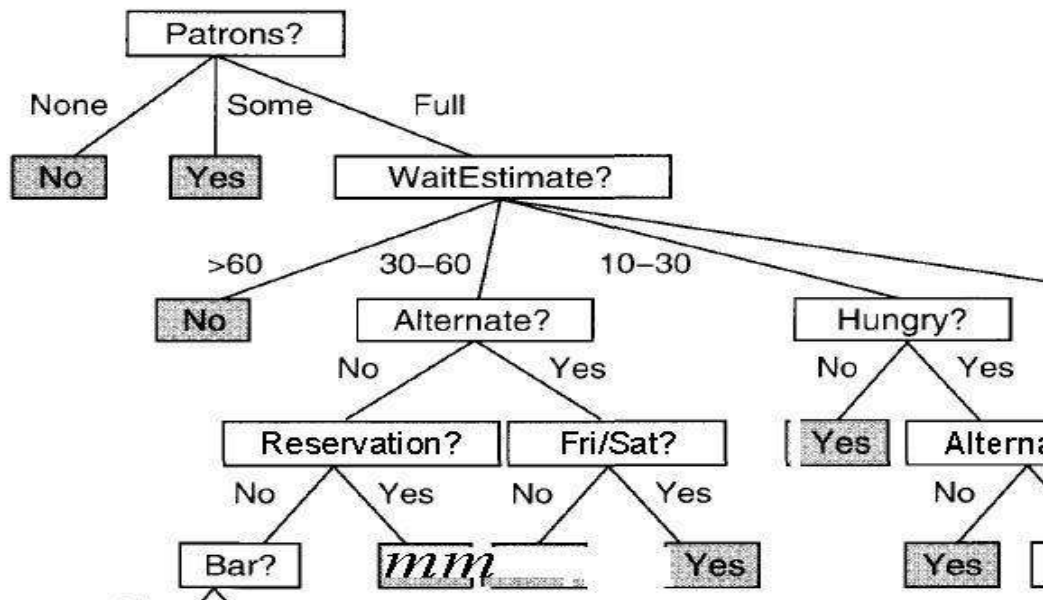
$\forall r \text{ Patrons}(r, \text{Full}) \wedge \text{WaitEstimate}(r, 10-30) \wedge \neg \text{Hungry}(r) \Rightarrow \text{WillWait}(r)$

27. What are Parity Function & Majority Function?

A) Parity Function is a function which returns 1 if and only if an even number of inputs are 1, then an exponentially large decision tree will be needed.

B) Majority function is one which returns 1 if more than half of its inputs are 1.

28. Draw an example decision tree



29. Explain the terms Positive example, negative example and training set.

An **example** is described by the values of the attributes and the value of the goal predicate. We call the value of the goal predicate the **classification** of the example. If the goal predicate is true for some example, we call it a **positive** example; otherwise we call it a **negative** example. A set of examples $X_1, \dots, X_n$ for the restaurant domain. The positive examples are ones where the goal *Will Wait* is true ($X_1, X_i, \dots$) and negative examples are ones where it is false ($X_2, X_5, \dots$). The complete set of examples is called the **training set**.

30. Explain the methodology used for accessing the performance of learning algorithm.

A Collect a large set of examples.

B Divide it into two disjoint sets: the **training set** and the **test set**.

X Use the learning algorithm with the training set as examples to generate a hypothesis

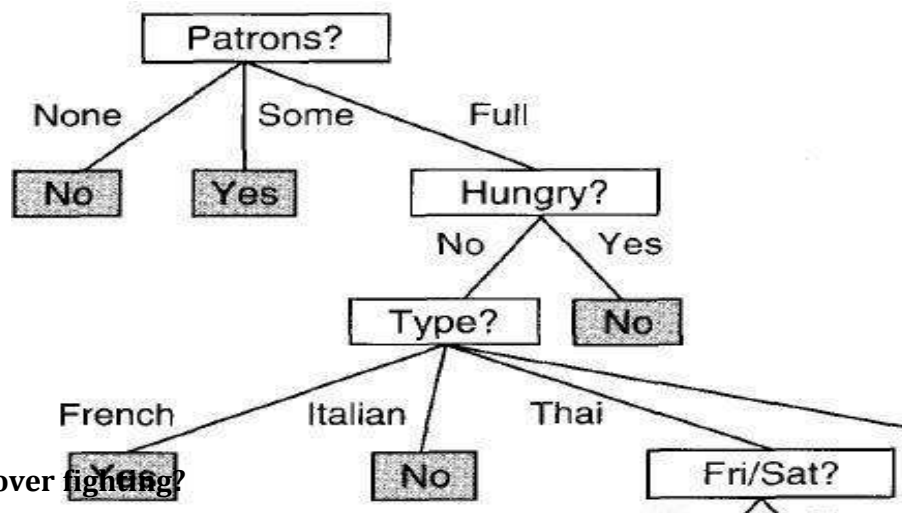
H .

Δ Measure the percentage of examples in the test set that are correctly classified by H .

E Repeat steps 1 to 4 for different sizes of training sets and different randomly selected

Φ Training sets of each size.

31. Draw an example of training set.



32. What is over fitting?

Whenever there is a large set of possible hypotheses, one has to be careful not to use the resulting freedom to find meaningless "regularity" in the data. This problem is called over fitting. It is a very general phenomenon, and occurs even when the target function is not at all random. It afflicts every kind of learning algorithm, not just decision trees.

33. What is Decision tree pruning?

Pruning works by preventing recursive splitting on attributes that are not clearly relevant, even when the data at that node in the tree is not uniformly classified.

34. What is Pruning?

The probability that the attribute is really irrelevant can be calculated with the help of standard statistical software. This is called pruning.

35. What is Cross Validation?

Cross-validation is another technique that eliminates the dangers of over fitting. The basic idea of cross-validation is to try to estimate how well the current hypothesis will predict unseen data. This is done by setting aside some fraction of the known data, and using it to test the prediction performance of a hypothesis induced from the rest of the known data.

36. What is Missing Data?

In many domains, not all the attribute values will be known for every example. The values may not have been recorded, or they may be too expensive to obtain. This gives rise to two problems. First, given a complete decision tree, how should one classify an object that is missing one of the test attributes? Second, how should one modify the information gain formula when some examples have unknown values for the attribute?

37. What are multivalued attributes?

When an attribute has a large number of possible values, the information gain measure gives an inappropriate indication of the attribute's usefulness. Consider the extreme case where every example has a different value for the attribute—for instance, if we were to use an attribute *Restaurant Name* in the restaurant domain. In such a case, each subset of examples is a singleton and therefore has a unique classification, so the information gain measure would have its highest value for this attribute. However, the GAIN RATIO attribute may be irrelevant or useless.

38. What is continuous valued attribute?

Attributes such as *Height* and *Weight* have a large or infinite set of possible values. They are therefore not well-suited for decision-tree learning in raw form. An obvious way to deal with this problem is to **discretize** the attribute. For example, the *Price* attribute for restaurants was discretized into \$, \$\$, and \$\$\$ values. Normally, such discrete ranges would be defined by hand. A better approach is to preprocess the raw attribute values during the tree-growing process in order to find out which ranges give the most useful information for classification purposes.

39. What are version space methods?

Version space methods are probably not practical in most real-world learning problems, mainly because of noise, they provide a good deal of insight into the logical structure of hypothesis space.

40. What is PAC learning?

Any hypothesis that is seriously wrong will almost certainly be "found out" with high probability after a small number of examples, because it will make an incorrect prediction. Thus, any hypothesis that is consistent with a sufficiently large set of training examples is unlikely to be seriously wrong—that is, it must be **Probably Approximately Correct**. **PAC-learning** is the subfield of computational learning theory that is devoted to this idea.

41. Difference between learning and performance agent?

Learning agents can be divided conceptually into a **performance element**, which is responsible for selecting actions, and a **learning element**, which is responsible for modifying the performance element.

42. Give a function for decision list learning?

function DECisiON-LiST-LEARNiNG(*e.ra;wp/<?.v*) **returns** a decision list, *No* or

failure **if** *examples* is empty **then return** the value *No* **t** — a test that matches a nonempty subset *examples*, of *examples* such that the members of *examples*, are all positive or all negative if there is no such *t* **then return** failure

if the examples in *examples*, are positive **then** *o* $\leftarrow$ *Yes*

else *o* $\leftarrow$ *No*

return a decision list with initial test / and outcome *o* and remaining elements given by DECISION-LIST-LEARNING

43. What is decision list?

A **decision list** is a logical expression of a restricted form. It consists of a series of tests, each of which is a conjunction of literals. If a test succeeds when applied to an example description, the decision list specifies the value to be returned. If the test fails, processing continues with the next test in the list.

44. What is state space search ?(Apr-May'17) (Apr-May'16)

State space search is a process used in the field of computer science, including artificial intelligence (AI), in which successive configurations or **states** of an instance are considered, with the goal of finding a goal **state** with a desired property.

45. How could you differentiate normal and decision tree? (Apr-May'17)

Decision Tree is a flow-chart with only if then else statements, drawn using insights from data. The process of making a decision tree involves extracting out if-else divisions (and their order) greedily such that the total entropy (or some other measure) of the leaves is lesser than the root.

Whereas Normal **Tree** nodes have many useful properties. The depth of a node is the length of the path (or the number of edges) from the root to that node. The height of a node is the longest path from that node to its leaves Binary: Each node has zero, one, or two children.

46. What is inductive Learning ?

Inductive learning is a kind of learning in which, given a set of examples an agent tries to estimate or create an evaluation function. Most inductive learning is supervised learning, in which examples provided with classifications. (The alternative is *clustering*.) More formally, an *example* is a pair $(x, f(x))$, where x is the input and $f(x)$ is the output of the function applied to x . The task of *pure inductive inference* (or *induction*) is, given a set of examples of f , to find a hypothesis h that approximates f .

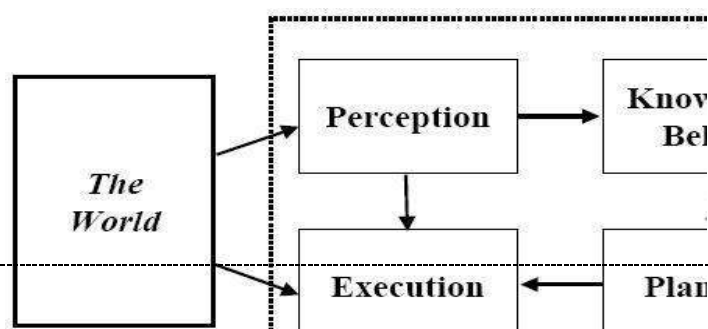
11 Marks

1. What is planning in AI? Explain the Planning done by an agent? (Nov'13) (Apr'15)

Planning problem

- Find a **sequence of actions** that achieves a given **goal** when executed from a given **initial world state**. That is, given
 - a set of operator descriptions (defining the possible primitive actions by the agent),
 - an initial state description, and
 - a goal state description or predicate, compute a plan, which is
 - a sequence of operator instances, such that executing them in the initial state will change the world to a state satisfying the goal-state description.
- Goals are usually specified as a conjunction of goals to be achieved

An Agent Architecture



Planning vs. problem solving

- Planning and problem solving methods can often solve the same sorts of problems
- Planning is more powerful because of the representations and methods used
- States, goals, and actions are decomposed into sets of sentences (usually in first-order logic)
- Search often proceeds through *plan space* rather than *state space* (though there are also state-space planners)
- Sub goals can be planned independently, reducing the complexity of the planning problem

Typical assumptions

- Atomic time: Each action is indivisible
- No concurrent actions are allowed (though actions do not need to be ordered with respect to each other in the plan)
- Deterministic actions: The result of actions are completely determined—there is no uncertainty in their effects
- Agent is the sole cause of change in the world
- Agent is omniscient: Has complete knowledge of the state of the world
- Closed World Assumption: everything known to be true in the world is included in the state description. Anything not listed is false.

Blocks world

The **blocks world** is a micro-world that consists of a table, a set of blocks and a robot hand. Some domain constraints:

- Only one block can be on another block
- Any number of blocks can be on the table
- The hand can only hold one block

Typical representation:

ontable(a)

ontable(c)

on(b,a) handempty clear(b) clear(c)

General Problem Solver

- The General Problem Solver (GPS) system was an early planner (Newell, Shaw, and Simon)
- GPS generated actions that reduced the difference between some state and a goal state
- GPS used Means-Ends Analysis
 - Compare what is given or known with what is desired and select a reasonable thing to do next
 - Use a table of differences to identify procedures to reduce types of differences
- GPS was a state space planner: it operated in the domain of state space problems specified by an initial state, some goal states, and a set of operations

Situation calculus planning

75. Intuition: Represent the planning problem using first-order logic

- Situation calculus lets us reason about changes in the world
- Use theorem proving to “prove” that a particular sequence of actions, when applied to the situation characterizing the world state, will lead to a desired result Situation calculus
- **Initial state:** a logical sentence about (situation) S_0
 $At(Home, S_0) \wedge \sim Have(Milk, S_0) \wedge \sim Have(Bananas, S_0) \wedge \sim Have(Drill, S_0)$
- **Goal state:**
 $(\exists s) At(Home, s) \wedge Have(Milk, s) \wedge Have(Bananas, s) \wedge Have(Drill, s)$
- **Operators** are descriptions of how the world changes as a result of the agent’s actions:
 $\forall (a, s) Have(Milk, Result(a, s)) \iff ((a = Buy(Milk) \wedge At(Grocery, s)) \vee (Have(Milk, s) \wedge a \neq Drop(Milk)))$
- $Result(a, s)$ names the situation resulting from executing action a in situation s .
- Action sequences are also useful: $Result'(l, s)$ is the result of executing the list of actions

(l) starting in s :

$$(\forall s) Result'([], s) = s$$

$$(\forall a, p, s) Result'([a|p]s) = Result'(p, Result(a, s))$$

Basic representations for planning

- Classic approach first used in the **STRIPS** planner circa 1970
- States represented as a conjunction of ground literals
 - $at(Home) \wedge \sim have(Milk) \wedge \sim have(bananas) \dots$
- Goals are conjunctions of literals, but may have variables which are assumed to be existentially quantified
 - $at(?x) \wedge have(Milk) \wedge have(bananas) \dots$

- Do not need to fully specify state
 - Non-specified either don't-care or assumed false
 - Represent many cases in small storage
 - Often only represent changes in state rather than entire situation
- Unlike theorem prover, not seeking whether the goal is true, but is there a sequence of actions to attain it

Operator/action representation

- Operators contain three components:
 - **Action description**
 - **Precondition** - conjunction of positive literals
 - **Effect** - conjunction of positive or negative literals which describe how situation changes when operator is applied

Example:

Op[Action: Go(there),
 Precond: At(here) ^ Path(here,there),
 Effect: At(there) ^ ~At(here)]

- All variables are universally quantified
- Situation variables are implicit
 - preconditions must be true in the state immediately before operator is applied;
 - effects are true immediately after

Blocks world operators

Here are the classic basic operations for the blocks world:

- stack(X,Y): put block X on block Y
- unstack(X,Y): remove block X from block Y
- pickup(X): pickup block X

- putdown(X): put block X on the table. Each will be represented by
 - a list of preconditions
 - a list of new facts to be added (add-effects)
 - a list of facts to be removed (delete-effects)
 - optionally, a set of (simple) variable constraints

For example:

```
preconditions(stack(X,Y), [holding(X),clear(Y)])
deletes(stack(X,Y), [holding(X),clear(Y)]).
adds(stack(X,Y), [handempty,on(X,Y),clear(X)])
constraints(stack(X,Y),
[X\==Y,Y\==table,X\==table])
```

STRIPS planning

- STRIPS maintain two additional data structures:
 - **State List** - all currently true predicates.
 - **Goal Stack** - a push down stack of goals to be solved, with current goal on top of stack.
- If current goal is not satisfied by present state, examine add lists of operators, and push operator and preconditions list on stack. (Sub goals)
- When a current goal is satisfied, POP it from stack.
- When an operator is on top stack, record the application of that operator on the plan sequence and use the operator's add and delete lists to update the current state.

Typical BW planning problem Initial state:

clear(a)

clear(b)

clear(c)

ontable(a)

ontable(b)

ontable(c) handempty

Goal interaction

- Simple planning algorithms assume that the goals to be achieved are independent
 - Each can be solved separately and then the solutions concatenated
- This planning problem, called the “Sussman Anomaly,” is the classic example of the goal interaction problem:
 - Solving on(A,B) first (by doing unstack(C,A), stack(A,B) will be undone when solving the second goal on(B,C) (by doing unstack(A,B), stack(B,C)).
 - Solving on(B,C) first will be undone when solving on(A,B)
- Classic STRIPS could not handle this, although minor modifications can get it to do simple cases

State-space planning

- We initially have a space of situations (where you are, what you have, etc.)
- The plan is a solution found by “searching” through the situations to get to the goal

- A **progression planner** searches forward from initial state to goal state
- A **regression planner** searches backward from the goal
 - This works if operators have enough information to go both ways
 - Ideally this leads to reduced branching –you are only considering things that are relevant to the goal Plan-space planning
- An alternative is to **search through the space of *plans***, rather than situations.
- Start from a **partial plan** which is expanded and refined until a complete plan that solves the problem is generated.
- **Refinement operators** add constraints to the partial plan and modification operators for other changes.
- We can still use STRIPS-style operators:

Op(ACTION: RightShoe, PRECOND: RightSockOn, EFFECT:
RightShoeOn) Op(ACTION: RightSock, EFFECT: RightSockOn)

Op(ACTION: LeftShoe, PRECOND: LeftSockOn, EFFECT: LeftShoeOn)

Op(ACTION: LeftSock, EFFECT:
leftSockOn) could result in a partial
plan of

[RightShoe, LeftShoe]

2. What is learning and its representation in AI explain?

Learning is an important area in AI, perhaps more so than planning.

- Problems are hard -- harder than planning.
- Recognized Solutions are not as common as planning.
- A goal of AI is to enable computers that can be taught rather than programmed.

Learning is an area of AI that focusses on processes of self-improvement.

Information processes that improve their performance or enlarge their knowledge bases are said to *learn*.

Why is it hard?

- Intelligence implies that an organism or machine must be able to adapt to new situations.
- It must be able to learn to do new things.
- This requires knowledge acquisition, inference, updating/refinement of knowledge base, acquisition of heuristics, applying faster searches, *etc.*

How can we learn?

Many approaches have been taken to attempt to provide a machine with learning capabilities. This is because learning tasks cover a wide range of phenomena.

Listed below are a few examples of how one may learn. We will look at these in detail shortly

Skill refinement

-- One can learn by practicing, *e.g. playing the piano*.

Knowledge acquisition

-- One can learn by experience and by storing the experience in a knowledge base. One basic example of this type is rote learning.

Taking advice

-- Similar to rote learning although the knowledge that is input may need to be transformed (or *operationalized*) in order to be used effectively.

Problem Solving

-- If we solve a problem one may learn from this experience. The next time we see a similar problem we can solve it more efficiently. This does not usually

involve gathering new knowledge but may involve reorganization of data or remembering how to achieve to solution.

Induction

-- One can learn from *examples*. Humans often classify things in the world without knowing explicit rules. Usually involves a teacher or trainer to aid the classification.

Discovery



Here one learns knowledge without the aid of a teacher.

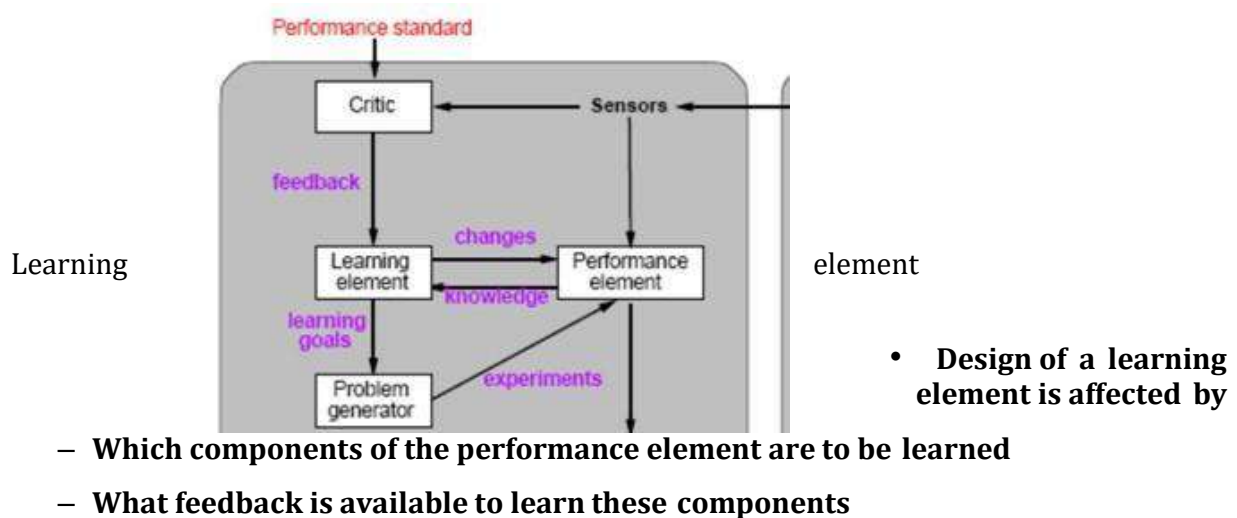
Analogy



If a system can recognize similarities in information already stored then it may be able to transfer some knowledge to improve to solution of the task in hand.

General model:

- **Learning is essential for unknown environments, i.e., when designer lacks omniscience. Learning is useful as a system construction method, i.e., expose the agent to reality rather than trying to write it down.**
- **Learning modifies the agent's decision mechanisms to improve performance**



- What representation is used for the components
- Type of feedback:
 - Supervised learning: correct answers for each example
 - Unsupervised learning: correct answers not given
 - Reinforcement learning: occasional rewards

3. Explain Inductive Learning

Inductive learning is a kind of learning in which, given a set of examples an agent tries to estimate or create an evaluation function. Most inductive learning is supervised learning, in which examples provided with classifications. (The alternative is *clustering*.) More formally, an *example* is a pair $(x, f(x))$, where x is the input and $f(x)$ is the output of the function applied to x . The task of *pure inductive inference* (or *induction*) is, given a set of examples of f , to find a hypothesis h that approximates f .

- Given an example, A pair $\langle x, f(x) \rangle$ where x is the input and $f(x)$ is the result
- Generate a hypothesis, A function $h(x)$ that approximates $f(x)$
- That will generalize well, Correctly predict values for unseen samples

How do we do this?

Must determine a hypothesis space...

- The set of all hypotheses we are willing to consider
- e.g. All functions of degree less than 10

That is realizable, contains the true function

Inductive learning method

Simplest form: learn a function from examples (tabula rasa)

f is the target function

O	O	X
	X	
X		

An example is a pair $x, f(x)$, e.g.,

Problem: find a(n) hypothesis h

such that $h \approx f$

given a training set of examples

(This is a highly simplified model of real learning:

{ Ignores prior knowledge

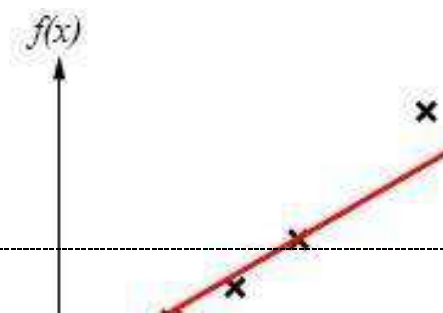
{ Assumes a deterministic, observable

\environment" { Assumes examples are given

{ Assumes that the agent wants to learn f [why?])

- **Construct/adjust h to agree with f on training set (h is consistent if it agrees with f on all examples)**
- **E.g., curve fitting:**

- **Construct/adjust h to agree with f on training set (h is consistent if it agrees with f on all examples)**
- **E.g., curve fitting:**



- Construct/adjust h to agree with f on training set (h is consistent if it agrees with f on all examples)
- E.g., curve fitting:



- Construct/adjust h to agree with f on training set (h is consistent if it agrees with f on all examples)
- E.g., curve fitting:

The most likely hypothesis is the simplest one consistent with the data."

Since there can be noise in the measurements, in practice we need to make a tradeoff between simplicity of the hypothesis and how well it fits the data.

5. Explain Reinforcement Learning

As opposed to *supervised learning*, reinforcement learning takes place in an environment where the agent cannot directly compare the results of its action to a desired result. Instead, it is given some *reward* or *punishment* that relates to its actions. It may win or lose a game, or be told it has made a good move or a poor one. The job of reinforcement learning is to find a successful function using these rewards.

The reason reinforcement learning is harder than supervised learning is that the agent is never told what the right action is, only whether it is doing well or poorly,

and in some cases (such as chess) it may only receive feedback after a long string of actions.

There are two basic kinds of information an agent can try to learn.

- **Utility function** -- The agent learns the utility of being in various states, and chooses actions to maximize the expected utility of their outcomes. This requires the agent keep a model of the environment.
- **Action-value** -- The agent learns an action-value function giving the expected utility of performing an action in a given state. This is called *Q-learning*. This is the *model-free* approach.

Passive Learning in a Known Environment

Assuming an environment consisting of a set of states, some terminal and some nonterminal, and a model that specifies the probabilities of transition from state to state, an agent learns passively by observing a set of *training sequences*, which consist of a set of state transitions followed by a reward.

The goal is to use the reward information to learn the expected utility of each of the nonterminal states. **An important simplifying assumption is that the utility of a sequence is the sum of the rewards accumulated in the states of the sequence.**

That is, the utility function is *additive*.

A passive learning agent keeps an estimate U of the utility of each state, a table N of how many times each state was seen, and a table M of transition probabilities. There are a variety of ways the agent can update its table U .

Naive Updating

One simple updating method is the *least mean squares* (LMS) approach [Widrow and Hoff, 1960]. It assumes that the observed reward-to-go of a state in a sequence provides direct evidence of the actual reward-to-go. The approach is simply to keep

the utility as a running average of the rewards based upon the number of times the state has been seen. This approach minimizes the mean square error with respect to the observed data.

This approach converges very slowly, because it ignores the fact that **the actual utility of a state is the probability-weighted average of its successors' utilities, plus its own reward**. LMS disregards these probabilities.

Adaptive Dynamic Programming

If the transition probabilities and the rewards of the states are known (which will usually happen after a reasonably small set of training examples), then the actual utilities can be computed directly as

$$U(i) = R(i) + \sum_j M_{ij}U(j)$$

where $U(i)$ is the utility of state i , R is its reward, and M_{ij} is the probability of transition from state i to state j . This is identical to a single *value determination* in the policy iteration algorithm for Markov decision processes. *Adaptive dynamic programming* is any kind of reinforcement learning method that works by solving the utility equations using a dynamic programming algorithm. It is exact, but of course highly inefficient in large state spaces.

Temporal Difference Learning

[Richard Sutton] Temporal difference learning uses the difference in utility values between successive states to adjust them from one epoch to another. The key idea is to use the observed transitions to adjust the values of the observed states so that they agree with the ADP constraint equations. Practically, this means updating the utility of state i so that it agrees better with its successor j . This is done with the *temporal-difference* (TD) equation:

$$U(i) \leftarrow U(i) + a(R(i) + U(j) - U(i))$$

where a is a *learning rate* parameter.

Temporal difference learning is a way of approximating the ADP constraint equations without solving them for all possible states. The idea generally is to define conditions that hold over local transitions when the utility estimates are correct, and then create update rules that nudge the estimates toward this equation.

This approach will cause $U(i)$ to converge to the correct value if the learning rate parameter decreases with the number of times a state has been visited [Dayan, 1992]. In general, as the number of training sequences tends to infinity, TD will converge on the same utilities as ADP.

Passive Learning in an Unknown Environment

Since neither temporal difference learning nor LMS actually use the model M of state transition probabilities, they will operate unchanged in an unknown environment. The ADP approach, however, updates its estimated model of an unknown environment after each step, and this model is used to revise the utility estimates.

Any method for learning stochastic functions can be used to learn the environment model; in particular, in a simple environment the transition probability M_{ij} is just the percentage of times state i has transitioned to j .

The basic difference between TD and ADP is that TD adjusts a state to agree with the observed successor, while ADP makes a state agree with all successors that might occur, weighted by their probabilities. More importantly, ADP's adjustments may need to be propagated across all of the utility equations, while TD's affect only the current equation. TD is essentially a crude first approximation to ADP.

A middle-ground can be found by bounding or ordering the number of adjustments made in ADP, beyond the simple one made in TD. The *prioritized-sweeping* heuristic prefers only to make adjustments to states whose *likely* successors have just undergone *large* adjustments in their utility estimates. Such approximate ADP

systems can be very nearly as efficient as ADP in terms of convergence, but operate much more quickly.

Active Learning in an Unknown Environment

The difference between active and passive agents is that passive agents learn a fixed policy, while the active agent must decide what action to take and how it will affect its rewards. To represent an active agent, the environment model M is extended to give the probability of a transition from a state i to a state j , *given an action a* . Utility is modified to be the reward of the state plus the maximum utility expected depending upon the agent's action:

$$U(i) = R(i) + \max_a \sum_j M_{ij}(a) U(j)$$

An ADP agent is extended to learn transition probabilities given actions; this is simply another dimension in its transition table. A TD agent must similarly be extended to have a model of the environment.

Learning Action-Value Functions

An action-value function assigns an expected utility to the result of performing a given action in a given state. If $Q(a, i)$ is the value of doing action a in state i , then

$$U(i) = \max_a Q(a, i)$$

The equations for Q-learning are similar to those for state-based learning agents. The difference is that Q-learning agents do not need models of the world. The equilibrium equation, which can be used directly (as with ADP agents) is

$$Q(a, i) = R(i) + \sum_j M_{ij}(a) \max_{a'} Q(a', j)$$

The temporal difference version does not require that a model be learned; its update equation is

$$Q(a, i) \leftarrow Q(a, i) + \alpha(R(i) + \max_{a'} Q(a', j) - Q(a, i))$$

Applications of Reinforcement Learning

The first significant reinforcement learning system was used in Arthur Samuel's checker-playing program. It used a weighted linear function to evaluate positions, though it did not use observed rewards in its learning process.

TD-gammon [Tesauro, 1992] has an evaluation function represented by a fully-connected neural network with one hidden layer of 80 nodes; with the inclusion of some precomputed board features in its input, it was able to reach world-class play after about 300,000 training games.

A case of reinforcement learning in robotics is the famous *cart-pole* balancing problem. The problem is to control the position of the cart (along a single axis) so as to keep a pole balanced on top of it upright, while staying within the limits of the track length. Actions are usually to jerk left or right, the so-called *bang-bang control* approach. The first work on this problem was the BOXES system [Michie and Chambers, 1968], in which state space was partitioned into boxes, and reinforcement propagated into the boxes. More recent simulated work using neural networks [Furuta et al., 1984] simulated the *triple-inverted-pendulum* problem, in which three poles balance one atop another on a cart.

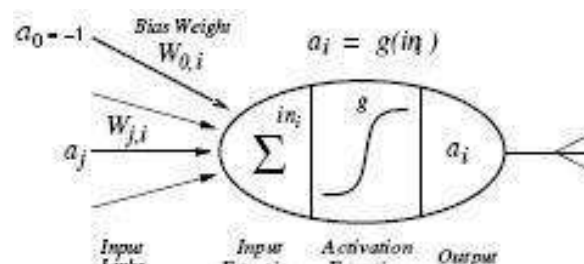
6. Discuss Neural Net learning and Genetic learning. (Apr/May'17)

Neural Networks

More specifically, a neural network consists of a set of *nodes* (or *units*), *links* that connect one node to another, and *weights* associated with each link. Some nodes receive inputs via links; others directly from the environment, and some nodes send outputs out of the network. Learning usually occurs by adjusting the weights on the links.

Each unit has a set of weighted inputs, an *activation level*, and a way to compute its activation level at the next time step. This is done by applying an *activation function* to the weighted sum of the node's inputs. Generally, the weighted sum (also called the *input function*) is a strictly linear sum, while the activation function may be nonlinear. If the value of the activation function is above a *threshold*, the node "fires."

Generally, all nodes share the same activation function and threshold value, and only the topology and weights change.



- g is a non-linear function which takes as input a weighted sum of the input link signals (as well as an intrinsic bias weight) and outputs a certain signal strength.
 - g is commonly a threshold function or sigmoid function.
- Network Structures

The two fundamental types of network structure are *feed-forward* and *recurrent*. A feed-forward network is a directed acyclic graph; information flows in one direction only, and there are no cycles. Such networks cannot represent internal state. Usually, neural networks are also *layered*, meaning that nodes are organized into groups of layers, and links only go from nodes to nodes in adjacent layers.

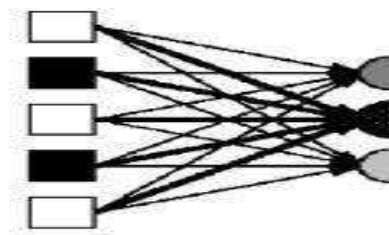
Recurrent networks allow loops, and as a result can represent state, though they are much more complex to analyze. *Hopfield networks* and *Boltzmann machines* are examples of recurrent networks; Hopfield networks are the best understood. All connections in Hopfield networks are bidirectional with symmetric weights, all units have outputs of 1 or -1, and the activation function is the sign function. Also, all nodes

in a Hopfield network are both input and output nodes. Interestingly, it has been shown that a Hopfield network can reliably recognize $0.138N$ training examples, where N is the number of units in the network.

Boltzmann machines allow non-input/output units, and they use a stochastic evaluation function that is based upon the sum of the total weighted input. Boltzmann machines are formally equivalent to a certain kind of belief network evaluated with a stochastic simulation algorithm.

One problem in building neural networks is deciding on the initial topology, e.g., how many nodes there are and how they are connected. Genetic algorithms have been used to explore this problem, but it is a large search space and this is a computationally intensive approach. The *optimal brain damage* method uses information theory to determine whether weights can be removed from the network without loss of performance, and possibly improving it. The alternative of making the network larger has been tested with the *tiling algorithm* [Mezard and Nadal, 1989] which takes an approach similar to induction on decision trees; it expands a unit by adding new ones to cover instances it misclassified. *Cross-validation* techniques can be used to determine when the network size is right.

Perceptrons



Perceptrons are single-layer, feed-forward networks that were first studied in the 1950's. They are only capable of learning *linearly separable* functions. That is, if we view F features as defining an F -dimensional space, the network can recognize any class that involves placing a single hyper plane between the instances of two classes. So, for example, they can easily represent **AND**, **OR**, or **NOT**, but cannot represent **XOR**.

Perceptron's learn by updating the weights on their links in response to the difference between their output value and the correct output value. The updating rule (due to Frank Rosenblatt, 1960) is as follows. Define *Err* as the difference between the correct output and actual output. Then the learning rule for each weight is

$$W_j \leftarrow W_j + A \times I_j \times \text{Err}$$

where *A* is a constant called the *learning rate*.

Of course, this was too good to last, and in *Perceptron's* [Minsky and Papert, 1969] it was observed how limited linearly separable functions were. Work on perceptron's withered, and neural networks didn't come into vogue again until the 1980's, when multi-layer networks became the focus.

```

function PERCEPTRON-LEARNING(examples, network) returns a perceptron hypothesis
  inputs: examples, a set of examples, each with input  $x = x_1, \dots, x_n$  and output  $y$ 
         network, a perceptron with weights  $W_j$ ,  $j = 0 \dots n$ , and activation function
  repeat
    for each  $e$  in examples do
       $\hat{m} \leftarrow \sum_{j=0}^n W_j x_j[e]$ 
       $\text{Err} \leftarrow y[e] - g(\hat{m})$ 
       $W_j \leftarrow W_j + \alpha \times \text{Err} \times g'(\hat{m}) \times x_j[e]$ 
  until some stopping criterion is satisfied

```

Multi-Layer Feed-Forward Networks

The standard method for learning in multi-layer feed-forward networks is *back-propagation* [Bryson and Ho, 1969]. Such networks have an input layer, and output layer, and one or more *hidden layers* in between. The difficulty is to divide the blame for an erroneous output among the nodes in the hidden layers.

The back-propagation rule is similar to the perceptron learning rule. If *Err_i* is the error at the output node, then the weight update for the link from unit *j* to unit *i* (the output node) is

$$W_{j,i} \leftarrow W_{j,i} + A \times a_j \times \text{Err}_i \times g'(in_i)$$

where g' is the derivative of the activation function, and a_j is the activation of the unit j . (Note that this means the activation function must have a derivative, so the sigmoid function is usually used rather than the step function.) Define D_i as $\text{Err}_i \times g'(in_i)$.

This updates the weights leading to the output node. To update the weights on the interior links, we use the idea that the hidden node j is responsible for part of the error in each of the nodes to which it connects. Thus the error at the output is divided according to the strength of the connection between the output node and the hidden node, and propagated backward to previous layers. Specifically,

$$D_j = g'(in_j) \times \sum_i W_{j,i} D_i$$

Thus the updating rule for internal nodes is

$$W_{j,i} \leftarrow W_{j,i} + A \times a_j \times D_i.$$

Lastly, the weight updating rule for the weights from the input layer to the hidden layer is

$$W_{k,j} \leftarrow W_{k,j} + A \times I_k \times D_j$$

where k is the input node and j the hidden node, and I_k is the input value of k .

A neural network requires $2^n/n$ hidden units to represent all Boolean functions of n inputs. For m training examples and W weights, each epoch in the learning process takes $O(mW)$ time; but in the worst case, the number of epochs can be exponential in the number of inputs.

In general, if the number of hidden nodes is too large, the network may learn only the training examples, while if the number is too small it may never converge on a set of weights consistent with the training examples.

Multi-layer feed-forward networks can represent any continuous function with a single hidden layer, and any function with two hidden layers [Cybenko, 1988, 1989].

Applications of Neural Networks

John Denker remarked that "neural networks are the second best way of doing just about anything." They provide passable performance on a wide variety of problems that are difficult to solve well using other methods.

NETtalk [Sejnowski and Rosenberg, 1987] was designed to learn how to pronounce written text. Input was a seven-character centered on the target character, and output was a set of Booleans controlling the form of the sound to be produced. It learned 95% accuracy on its training set, but had only 78% accuracy on the test set. Not spectacularly good, but important because it impressed many people with the potential of neural networks.

Other applications include a ZIP code recognition [Le Cun et al., 1989] system that achieves 99% accuracy on handwritten codes, and driving [Pomerleau, 1993] in the ALVINN system at CMU. ALVINN controls the NavLab vehicles, and translates inputs from a video image into steering control directions. ALVINN performs exceptionally well on the particular road-type

it learns, but poorly on other terrain types. The extended MANIAC system [Jochem et al., 1993] has multiple ALVINN subnets combined to handle different road types.

7. What is conditional planning? Explain. (Apr/May'14) (Nov'15) (Apr/May'17)

Conditional planning is a way to deal with uncertainty by checking what is in the environment at predetermined points in the plan. Conditional planning explain for fully observable environments, so we will begin with that case. A fully observable case is more difficult, but more interesting.

Conditional planning in fully observable environments

Full observability means that the agent always knows the current state. If the environment is nondeterministic, however, the agent will not be able to predict the outcome of its actions. A conditional planning agent handles nondeterminism by building into the plan (at execution time) conditional steps that will check the state of the environment (at execution time) and decide what to do next. The problem, then, is how to construct these conditional

The first thing we need to do is augment the STRIPS language

function AND-OR-GRAPH-SEARCH(*problem*) **returns** a conditional plan, or failure
 OR-SEARCH([INITIAL-STATE | *problem*], *problem*, [])

function OR-SEARCH(*state*, *problem*, *path*) **returns** a conditional plan, or failure
 if GOAL-TEST[*problem*](*state*) **then return** the empty plan
 if *state* is on *path* **then return** failure
 for each *action*, *state_set* in SUCCESSORS[*problem*](*state*) **do**
 plan ← AND-SEARCH(*state_set*, *problem*, [*state* | *path*])
 if *plan* ≠ failure **then return** [*action* | *plan*]
return failure

function AND-SEARCH(*state_set*, *problem*, *path*) **returns** a conditional plan, or failure
 for each *s_i* in *state_set* **do**
 plan_i ← OR-SEARCH(*s_i*, *problem*, *path*)
 if *plan* = failure **then return** failure
return [if *s₁* **then** *plan₁* **else if** *s₂* **then** *plan₂* **else** ... if *s_{n-1}* **then** *plan_{n-1}* **else** failure]

Conditional planning in partially observable environments

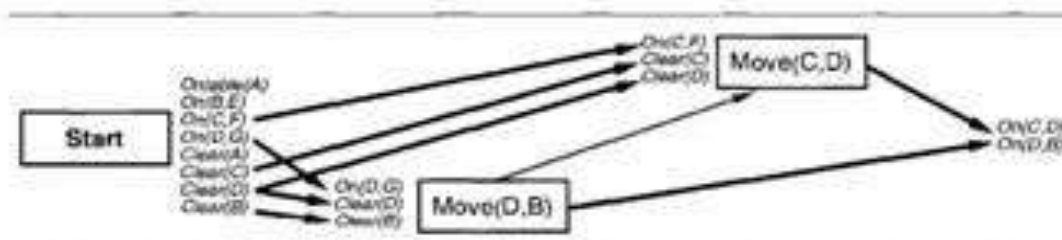
The preceding section dealt with fully observable environments, which have that conditional tests can ask any question at all and be sure of getting an answer. In the initial state of a partially observable environment, partial observability is much more common. In the initial state of a partially observable planning problem, the agent knows only a certain amount about the actual state. One way to model this situation is to say that the initial state belongs to a **state set**;

8. Describe continuous planning. (Apr/May'14) (Nov'15)

we design an agent that persists indefinitely in an environment, not a "problem solver" that is given a single goal and then plans and acts until the goal is achieved; rather, it lives through a series of ever-changing goal formulation and acting phases. Rather than thinking of the planner and execution monitoring processes, one of which passes its results to the other, we can think of the whole process in a **continuous planning agent**.

The agent is thought of as always being *part of the way through* executing a grand plan of living its life. Its activities include executing some steps of the plan that is ready to be executed, refining the plan to satisfy open preconditions or resolving conflicts, or modifying the plan in the light of additional information obtained during execution. Thus, when it first formulates a new goal, the agent will have no actions ready to be executed; it will spend a while generating a partial plan. It is quite possible, however, for

Throughout the continuous planning process, *Start* is used as the label for the current state. The agent updates the state after each action.



The initial plan constructed by the continuous planning agent. It is indistinguishable, so far, from that produced by a normal partial-order planner.

The plan is now ready to be executed, but before the agent can take action, an external agent (perhaps the agent's teacher getting impatient) moves block D onto block B, and the world is now in the state shown in Figure 12.15(b). The agent perceives this change and recognizes that *Clear(B)* and *On(D,G)* are no longer true in the current state, and updates its model of the current state accordingly. The causal links that were supplying the preconditions *Clear(B)* and *On(D,G)* for the *Move(D,B)* action become invalid and must be removed from the plan. The new plan is shown in Figure 12.17. At all times, *Start* represents the current state, so this *Start* is different from the one in the previous figure. Note that the plan is now incomplete: two of the preconditions for *Move(D,B)* are open, and in addition *On(D,y)* is now uninstantiated, because there is no longer any reason to think that move will be from G.

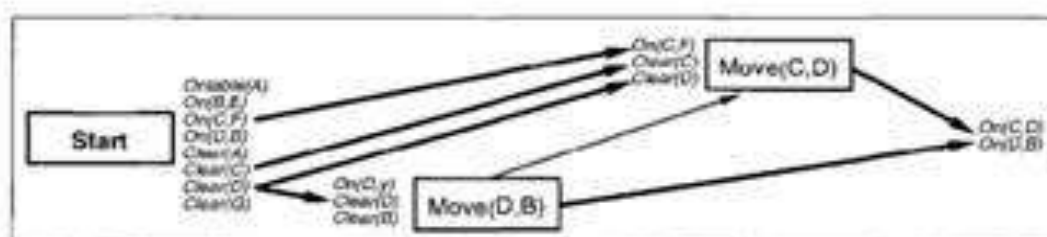


Figure 12.17 After someone else moves D onto B, the unsupported links (*Clear(B)* and *On(D,G)*) are deleted, rendering this plan incomplete.

algorithm where the two flaws are open preconditions and causal conflicts. If planning agent, on the other hand, addresses a much broader range of flaws:

- *Missing goal:* The agent can decide to add a new goal or goals to the (Under continuous planning, it might make more sense to change the name to *Infinity*, and of *Start* to *Current*, but we will stick with tradition.)
- *Open precondition:* Add a causal link to an open precondition, choosing or an existing action (as in POP).
- *Causal Conflict:* Given a causal link $A \xrightarrow{p} B$ and an action C with effect p , add an ordering constraint or variable constraint to resolve the conflict (as in POP).
- *Unsupported link:* If there is a causal link $Start \xrightarrow{p} A$ where p is not a precondition of $Start$, then remove the link. (This prevents us from executing an action when its preconditions are false.)
- *Redundant action:* If an action A supplies no causal links, remove it and it allows us to take advantage of serendipitous events.)
- *Unexecuted action:* If an action A (other than *Finish*) has its precondition in *Start*, has no other actions (besides *Start*) ordered before it, and contains no causal links, remove it.

9.Explain Multi agent Planning in detail .

Multi-agent systems consist of agents and their environment. Typically multi-agent systems research refers to software agents. However, the agents in a multi-agent system could equally well be robots, humans or human teams. A multi-agent system may contain combined human-agent teams.

Agents can be divided into different types ranging from simple to complex. Some categories suggested to define these types include:

- **Passive agents** or agent without goals (like obstacle, apple or key in any simple simulation)
- **Active agents** with simple goals (like birds in flocking, or wolf–sheep in prey-predator model)
- Cognitive agents (which contain complex calculations)

Agent environments can be divided into:

- Virtual Environment
- Discrete Environment
- Continuous Environment

Agent environments can also be organized according to various properties like: accessibility (depending on if it is possible to gather complete information about the environment), determinism (if an action performed in the environment causes a definite effect), dynamics (how many entities influence the environment in the moment), discreteness (whether the number of possible actions in

the environment is finite), episodicity (whether agent actions in certain time periods influence other periods), and dimensionality (whether spatial characteristics are important factors of the environment and the agent considers space in its decision making). Agent actions in the environment are typically mediated via an appropriate middleware. This middleware offers a first-class design abstraction for multi-agent systems, providing means to govern resource access and agent coordination.

Characteristics

The agents in a multi-agent system have several important characteristics:

- **Autonomy:** the agents are at least partially independent, self-aware, autonomous
- **Local views:** no agent has a full global view of the system, or the system is too complex for an agent to make practical use of such knowledge
- **Decentralisation:** there is no designated controlling agent (or the system is effectively reduced to a monolithic system)

Self-organisation and self-steering

Multi-agent systems can manifest self-organisation as well as self-steering and other control paradigms and related complex behaviors even when the individual strategies of all their agents are simple. When agents can share knowledge using any agreed language, within the constraints of the system's communication protocol, the approach may lead to a common improvement. Example languages are Knowledge Query Manipulation Language (KQML) or FIPA's Agent Communication Language (ACL).

QUESTION BANK & UNIVERSITY QUESTIONS

1. Write about:
 - a. Partial Order Planning
 - b. Conditional Planning (Nov'15)(Dec'17) (Pg.No. 30, Q.No. 7)
2. Discuss the syntax and semantics of associate network.
3. Briefly explain the basic ascepts of planning. (Nov'13) (Pg.No. 10, Q.No. 1)
4. (a) Is there any connection between learning and planning methods? Explain. (8)
(b) Explain the components of a replanning agent.
5. Describe how Decision Trees could be used for inductive learning. Explain its effectiveness with a suitable example. (Pg.No. 19, Q.No. 3)
6. Discuss Partial Order Planning in detail with an example.
7. Explain the process of planning with state space search with an example. (Apr'15)(April'18) (Pg.No. 10, Q.No. 1)
8. Explain decision free learning in detail with suitable examples. (Apr'15)
9. Discuss in detail about genetic algorithm with an example. (Nov'15) (Apr/May'16) (Apr'18)
10. (a) What is genetic algorithm? What are the advantages and disadvantages of Generic algorithm (Nov'15)(Dec'17)
(b) Describe the three major issues affected in the design of a learning agent. (Nov'15)
11. (a) What is learning? Explain learning by problem solving with suitable examples. (Pg.No. 16, Q.No. 2)
(b) Explain Rote learning in detail. (Nov/Dec'14)
12. Elaborate on Multi agent planning. (Nov/Dec'14)(Apr-May-16)
13. Discuss in detail about neural networks. (Pg.No. 26, Q.No. 6)
14. Discuss about learning decision trees. (Nov'13) (Apr/May'14)
15. Write about: (Apr/May'14) (Nov'15)
 - a. Continuous Planning (Pg.No. 32, Q.No. 8)
 - b. Conditional Planning (Pg.No. 30, Q.No. 7)
16. Explain about partial order planning algorithm with an example. (Nov'18)